

Data Mining

Deep Learning Project Handbook

Instructor: Dr. Hadi Farahani

Term: Spring 2026

Deliverable: Code + Report (+ optional Demo)

Deadline: August 5, 2026

In one line. Pick a personal path through eight projects across five deep-learning families. Reach **100 points** for a full score; every point above 100 is a bonus. Your 100 must span **at least three of the five architecture families**, and each project requires a genuine **original contribution of your own**—not just running a pretrained model. Two further **special bonus challenges** (Section 4.3) let the strongest students go further still.

Contents

1	Overview and Learning Goals	3
2	How Grading Works	3
3	Rules, Deliverables and Integrity	4
4	Project Catalog	5
5	Suggested Report Outline and Submission Checklist	13

1. Overview and Learning Goals

This handbook defines a menu of practical, hands-on projects for the five architectures we study this term: **Recurrent Neural Networks**, **Transformers**, **Vision Transformers**, **Autoencoders**, and **Variational Autoencoders**. Rather than one fixed assignment, you assemble your own program of work from the catalog in Section 4.

The projects are deliberately chosen to be useful and motivating—forecasting weather, detecting abnormal heartbeats, translating between languages, generating images, catching crop disease from a leaf photo. By the end you should be able to:

- take a real dataset from raw form to a working, evaluated model;
- choose the right architecture for a sequential, textual, visual, or generative problem;
- design and defend an *original* experimental contribution, not merely reproduce a tutorial;
- report results honestly, with appropriate baselines, metrics, and error analysis.

Every dataset referenced here has been checked and is freely accessible at the time of writing (Hugging Face Hub, torchvision, Keras utilities, Kaggle, or the UCR/UEA archive).

2. How Grading Works

Points and the road to 100

Each project carries a point value set by its difficulty. **Foundational** projects are small and self-contained; **Core** projects are the backbone of the course; **Advanced** projects are research-flavored and worth the most.

- You need **100 points** for a complete score.
- The catalog offers **235 points** in total, so you have real choice about what to build.
- **Any points beyond 100 count as bonus** and can lift your final mark above full credit.

Breadth rule — span at least three families Your first 100 points must come from projects covering **at least three of the five architecture families** (RNN, Transformer, Vision Transformer, Autoencoder, VAE). Note that the two Vision-Transformer projects count as *one* family, and likewise the two VAE projects. You cannot reach a complete score by grinding a single family.

Points by family

Family	Available projects	Points
RNN	Weather forecasting	25
Transformer	Text classification (25) · EN ↔ FA translation (35)	60
Vision Transformer	Fine-tune + CNN baseline (30) · ViT from scratch (35)	65
Autoencoder	ECG anomaly detection	20
VAE	Generative + latent space (30) · Conditional/ β -VAE (35)	65
Total available (counts toward the 100 and the breadth rule)		235

A third kind of bonus: special challenges Section 4.3 adds **two special bonus challenges** (25 points each, **50 points total**) proposed for students who want to demonstrate real depth: one links reinforcement learning to Transformers, the other asks you to redesign the attention mechanism itself. These sit **outside** the 235 above — they do not count toward the required 100 or toward the breadth rule, they are pure extra credit on top of everything else.

Example paths to a complete score

- **Breadth path:** Forecasting (25) + Text classification (25) + ViT fine-tune (30) + ECG (20) = **100**, four families.
- **Depth path:** ViT from scratch (35) + EN ↔ FA translation (35) + VAE generative (30) = **100**, three families.
- **Overachiever:** all five Core projects (130) + one Advanced (35) = **165**, i.e. 65 bonus points across all five families.

Demo bonus

A project shipped with a working demo (a short notebook walkthrough, a small Gradio/Streamlit app, or a live viva demonstration) earns +5 **bonus points**, up to +15 total across your submission. Demos are optional but strongly encouraged—they are the easiest bonus in the course and make your work memorable.

3. Rules, Deliverables and Integrity

The novelty requirement (read this carefully)

You may freely use pretrained models, Hugging Face, `timm`, PyTorch/TensorFlow, and any open library. **But a submission that only runs someone else's trained model receives no credit.** Each project below specifies a *required original contribution*: something you design, implement, and evaluate yourself. Acceptable forms of novelty include:

- a proposed architectural modification, custom loss, or custom scoring rule, with an ablation;
- a controlled comparison built around a stated hypothesis (not a generic “A vs B”);
- applying a method to a new domain/dataset and analyzing precisely what breaks and why;
- a targeted, evidence-backed fix for a failure mode you first diagnose.

Rule of thumb. If your entire project could be reproduced by copying a public tutorial and swapping the dataset path, it is not enough. We grade the *contribution and the analysis*, not the accuracy number alone.

What to submit

1. **Code** — a clean repository with a README, pinned dependencies, fixed random seeds, and instructions to reproduce your main result end-to-end.
2. **Report** — a concise technical write-up (suggested outline in Section 5): problem, data, method, *your original contribution*, experiments, results, honest analysis, and limitations.
3. **Discussion & observation (required)** — for *every* project you attempt, your report must include a short section answering that project's **Discussion & observation** questions (the blue box on each card). These are graded on the quality of your reasoning, not on reaching a “correct” answer: we are looking for sharp observation, honest use of your own results, and a willingness

to speculate carefully. Treat them as a core part of the project, worth roughly one fifth of its marks — a project with a strong implementation but thin discussion will not receive full credit.

4. **Demo (optional, bonus)** — notebook, small app, or live demonstration.

Teams and logistics

Projects may be done **individually or in pairs** (*instructor to confirm*); a pair is expected to attempt proportionally more scope. Late policy, submission channel, and viva scheduling will be announced separately.

Academic integrity

Cite every model, dataset, and code source you build on. Using pretrained weights and libraries is expected; presenting others' work, text, or code as your own is not. When in doubt about whether something counts as your contribution, ask before the deadline.

4. Project Catalog

Each card lists the dataset and how to obtain it, the core task, the **required original contribution**, what to report, and optional stretch goals for extra bonus. Difficulty is marked as **FOUNDATIONAL**, **CORE**, **ADVANCED**, or (for the two special challenges in Section 4.3) **BONUS CHALLENGE**.

4.1 Core Projects

25 pts
CORE
RNN family

Dataset & access. Jena Climate (Max Planck Institute for Biogeochemistry): $\approx 420,551$ records, 14 weather sensors at 10-minute resolution. Bundled with Keras (`keras.utils.get_file`) and widely mirrored; a clean CSV is a single download.

Task. Forecast one or more variables (e.g. temperature, humidity) several steps ahead from a sliding window of past readings. Build and compare LSTM, GRU, and a simple sequence-to-sequence variant across increasing forecast horizons.

Required original contribution. Go beyond a stock recurrent model: propose and test at least one improvement of your own—an attention mechanism over the recurrent states, a decomposition-then-forecast pipeline, or a custom horizon-weighted loss—and demonstrate it moves MAE/RMSE against *your own* baseline. State the hypothesis first.

Report. Windowing choices, baseline vs. your model, how error compounds with horizon, and where the model fails (e.g. sudden fronts).

Discussion & observation (*answer these in your report*)

1. Attention was introduced partly to fix what recurrence struggles with. If you added an attention mechanism over past timesteps to your forecaster, what would it learn to “look back” at in a weather signal, and would you expect the gain to be larger at short or long horizons? Explain your reasoning.
2. A Transformer can forecast the same series with no recurrence at all. What does the RNN’s sequential inductive bias buy you that a Transformer lacks here, and in what regime (data size, sequence length) would you still bet on the RNN?

Stretch (bonus). Probabilistic forecasts with prediction intervals; multi-station generalization.

25 pts

CORE

Transformer family

Dataset & access. Choose one: **AG News** (topic), **GoEmotions** (27 emotions, Reddit), or **Jigsaw Toxic Comments**. All available on the Hugging Face Hub (datasets) and/or Kaggle. Backbone: DistilBERT / BERT via transformers.

Task. Fine-tune a pretrained encoder for the classification task and beat a from-scratch or classical (TF-IDF + linear) baseline you also build.

Required original contribution. A plain fine-tune-and-report is not sufficient. Add an original angle: a data-efficiency curve (few-shot fine-tuning), *full fine-tuning vs. a parameter-efficient method* (LoRA/adapters) you implement, a targeted augmentation, or an error-analysis-driven fix—each with before/after evidence.

Report. Baseline gap, your intervention, per-class metrics (mind class imbalance), and representative error cases.

Discussion & observation (*answer these in your report*)

1. A 1-D convolution over word embeddings and a self-attention layer both build sentence representations. What can convolution “see” that attention cannot, and vice versa? Give a sentence on which you would expect the two to disagree, and say why.
2. Could you place a convolutional layer inside a Transformer block (or attention inside a text-CNN)? Where would you insert it, and what would each mechanism contribute to the final representation?

Stretch (bonus). Calibration analysis; robustness to a distribution shift you construct.

30 pts

CORE

Vision Transformer family

Dataset & access. **PlantVillage** ($\approx 54,000$ leaf images, 38 crop-disease classes) or **Food-101** (101,000 images, 101 classes). Both on the Hugging Face Hub / torchvision / Kaggle. Pretrained ViT-B/16 from timm or transformers; ResNet baseline from torchvision.

Task. Fine-tune a pretrained ViT and a comparable CNN (e.g. ResNet-50) on the same split, under a matched training budget.

Required original contribution. Not just “which wins”—design an investigation: a per-class disagreement analysis between ViT and CNN, an attention-rollout interpretability probe, a robustness-to-corruption study, or a layer-unfreezing strategy comparison. State a hypothesis and support it quantitatively.

Report. Matched-budget results, where the two models disagree, and one interpretable insight (attention maps or Grad-CAM).

Discussion & observation (*answer these in your report*)

1. Convolution is local and translation-equivariant; self-attention is global. Design a hybrid that injects attention into a CNN (or convolution into a ViT). Would you place the attention early or late in the network, and why? What change in the learned features would you predict?
2. Put your ViT's attention maps and the CNN's activation / Grad-CAM maps side by side on the same image. What does each appear to focus on, and does the difference match your intuition about how the two “see” an image?

Stretch (bonus). Data-efficiency sweep (accuracy vs. training-set size) for ViT vs. CNN.

20 pts

FOUNDATIONAL

Autoencoder family

Dataset & access. **ECG5000** — 5,000 heartbeat sequences of length 140, labeled normal/abnormal. Available from the UCR/UEA archive (timeseriesclassification.com) and the Hugging Face Hub.

Task. Train an autoencoder on *normal* beats only, then flag anomalies by reconstruction error against a chosen threshold. Report precision/recall/F1 (the classes are imbalanced—accuracy alone is misleading).

Required original contribution. Propose your own scoring or architecture choice: a controlled convolutional-AE vs. LSTM-AE comparison, a non-trivial thresholding rule you justify (beyond a fixed percentile), or an analysis of *which* anomaly types are missed plus a fix. Show the effect on precision/recall.

Report. Threshold selection, error-distribution plots for normal vs. abnormal, and failure cases.

Discussion & observation (*answer these in your report*)

1. Reconstruction-based anomaly detection has a paradox: if the autoencoder becomes too good, it may reconstruct anomalies well and stop flagging them. Discuss this tension, and how you would deliberately limit the model (capacity, bottleneck size, training data) to keep it sensitive.
2. If you added self-attention to the encoder, what might the attention weights reveal about which part of a heartbeat carries the “normal” signature — and would attention genuinely help, or just add capacity?

Stretch (bonus). Compare against a one-class SVM or Isolation Forest baseline.

30 pts

CORE

VAE family

Dataset & access. **Fashion-MNIST** or **MNIST** for a fast start (torchvision); **CelebA** ($\approx 202,599$ faces, torchvision / Hugging Face) for the harder version.

Task. Train a VAE, sample new images, interpolate between two encodings, and visualize the latent space (2-D projection or a latent grid).

Required original contribution. Contribute something of your own to the latent space or generation quality: quantitatively evaluate latent-space arithmetic / attribute manipulation, propose and test a regularizer or a modified prior, or design a study of the reconstruction-vs-generation trade-off. “Train a VAE and show samples” alone will not earn full credit.

Report. Architecture, ELBO curves, sample quality discussion, and what structure the latent space learned.

Discussion & observation (*answer these in your report*)

1. Interpolating between two points in a VAE’s latent space yields smooth, meaningful images; the same interpolation in a plain autoencoder often yields garbage. Why? What is the KL term doing to the geometry of the latent space to make interpolation work?
2. Suppose you replaced the Gaussian prior with something else (a mixture, or a uniform ball). How would sampling and the shape of the latent space change, and what might break?

Stretch (bonus). Quantify sample quality (e.g. FID) rather than eyeballing.

4.2 Advanced Projects

Transformer family

35 pts

ADVANCED

Dataset & access. **Mizan** ($\approx 1\text{M}$ literary sentence pairs) or the **ParsiNLU** translation split (integrates Mizan, TEP, and Global Voices; $\approx 1.6\text{M}$ pairs), both on the Hugging Face Hub; OPUS EN–FA as an additional source. A low-resource, genuinely challenging pair.

Task. Train from scratch or fine-tune a Transformer for translation in one direction (or both), and evaluate with BLEU/chrF plus qualitative examples.

Required original contribution. This is research-flavored. Contribute an original element: a normalization pipeline for Persian script (Arabic-vs-Persian character variants, zero-width non-joiner handling), a subword-tokenization study, back-translation augmentation, or a structured failure-mode analysis (morphology, idioms) with a targeted improvement—measured against your own baseline.

Report. Preprocessing decisions, BLEU/chrF with your contribution ablated in/out, and a qualitative error taxonomy.

Discussion & observation (*answer these in your report*)

1. Attention gives a soft alignment between source and target words. Given that English and Persian differ in word order and morphology (and Persian is written right-to-left), what alignment structure would you expect the cross-attention matrix to show, and where would it look “messy”?
2. A purely convolutional seq2seq (no attention) can also translate. What specifically would it lose on long sentences, and why is attention such a natural fit for translation in particular?

Stretch (bonus). A working translate-your-own-sentence demo (counts toward demo bonus).

35 pts

ADVANCED

Vision Transformer family

Dataset & access. **CIFAR-10** (torchvision). Compare against a small CNN baseline (e.g. ResNet-18) trained under the same budget.

Task. Implement a Vision Transformer from scratch (patch embedding, multi-head self-attention, class token, positional embeddings) and establish its gap to a CNN on small-scale data.

Required original contribution. The deliverable is a controlled investigation of inductive bias. After establishing the gap, propose and test a modification that narrows it—a convolutional stem/tokenizer, a strong augmentation regime, or knowledge distillation from a CNN—and present the ablation that isolates its effect.

Report. Clean from-scratch implementation, the CNN-vs-ViT gap, and how far your modification closes it, with the ablation table.

Discussion & observation (*answer these in your report*)

1. The ViT–CNN gap on small data is usually blamed on “missing inductive bias.” Is that a fundamental property of self-attention, or an optimization/data artifact that more data or better training removes? Argue a position using your own results.
2. Design the *minimal* change that reintroduces locality into a ViT without abandoning self-attention. Why might a convolutional stem be enough, and what does that tell you about *where* in the network the bias actually needs to live?

Stretch (bonus). Attention-map visualization across depth; sensitivity to patch size.

35 pts

ADVANCED

VAE family

Dataset & access. **MNIST** / **Fashion-MNIST** (conditional generation, torchvision), or **dSprites** (the standard disentanglement benchmark, on GitHub) / **CelebA** with attribute labels.

Task. Extend a VAE into a Conditional VAE (generate a chosen class/attribute) or a β -VAE with an annealing schedule, and study control over the generated output.

Required original contribution. Go beyond a vanilla VAE: implement conditioning or a β /annealing scheme and evaluate it with a chosen disentanglement metric (e.g. MIG or a factor-score) or a controllability demo backed by *quantitative* evidence—not eyeballed samples.

Report. The conditioning/ β mechanism, the metric you chose and why, and evidence of control or disentanglement.

Discussion & observation (*answer these in your report*)

1. Raising β improves disentanglement but hurts reconstruction. Explain intuitively why forcing the latent code to carry *less* information encourages independent factors. Is there a link to an information-bottleneck or rate-distortion view?
2. “Disentanglement” is loosely defined. Propose your own informal test for whether a latent dimension is a single interpretable factor — and describe a case where disentanglement would actually be *undesirable*.

Stretch (bonus). Latent-traversal grids that visibly isolate single factors.

4.3 Special Bonus Challenges

These two are proposed on top of the regular catalog, for students who want to show real depth. They are **harder and more open-ended** than the Advanced projects above, and — unlike every project so far — their points do **not** count toward the required 100 or toward the breadth rule. They are pure extra credit.

25 pts

BONUS CHALLENGE

RL × Transformer

Dataset & access. IMDB movie reviews (stanfordnlp/imdb on the Hugging Face Hub) as prompts; a pretrained sentiment classifier (lvwerra/distilbert-imdb) as the reward signal. Backbone: GPT-2 (small). Training loop: Hugging Face’s `trl` library (PPOTrainer), which is built exactly for this — fine-tuning a Transformer language model with policy-gradient RL (PPO) against a scalar reward.

Task. Starting from GPT-2, use PPO to fine-tune the model so that its continuations of movie-review prompts are steered toward a target sentiment, using the classifier’s confidence on the generated text as the reward. This is a small, self-contained version of the same idea behind RLHF (reinforcement learning from human feedback), the technique used to align modern chat models.

Required original contribution. The standard `trl` tutorial reproduces this end-to-end, so that alone is not enough. Go further: study *reward hacking* by training with and without a KL-to-reference-model penalty and showing what happens to fluency (e.g. perplexity) as reward climbs; or make the model *controllably* bidirectional (steer to positive or negative on demand) and analyze failure cases; or compare PPO fine-tuning against a simple supervised baseline (fine-tune directly on classifier-labeled positive reviews) and discuss what each can and cannot do that the other can’t.

Report. The reward curve over training, qualitative before/after generations, the fluency-vs-reward trade-off, and what the KL penalty changed.

Discussion & observation (*answer these in your report*)

1. Supervised fine-tuning optimizes cross-entropy against fixed target text; PPO optimizes expected reward from a classifier with no fixed targets at all. What fundamentally different training signal does each provide, and why can pure reward maximization drift into fluent-looking gibberish or degenerate repetition without a KL penalty holding the policy near a reference model?
2. This setup is a miniature of how real chat models are aligned with RLHF. In that setting, what plays the role of the sentiment classifier, and what could go wrong — for the model's behavior — if that reward model is systematically wrong about what it's rewarding?

Stretch (bonus). Train your own tiny reward model from preference pairs you label yourself, instead of using the off-the-shelf classifier.

25 pts**BONUS CHALLENGE****Attention design**

Dataset & access. Build your own small OCR-style corpus by rendering random digit/character strings onto fixed-size grayscale images with a monospace font using Pillow (fully offline, no download, full control over image size) — or, for a public alternative with clear row/column digit structure, **SVHN** (`torchvision.datasets.SVHN`).

Task. Full 2-D self-attention over an $H \times W$ feature map costs $O((HW)^2)$ because every location attends to every other location at once. Wang et al., “*Axial-DeepLab: Stand-Alone Axial-Attention for Panoptic Segmentation*” (ECCV 2020), factorize this into two cheap 1-D attentions: a **row-attention** (each position attends across its row) followed by a **column-attention** (each position attends across its column), reducing the cost to $O(HW(H + W))$ — exactly the row/column structure an OCR-like image has. Implement this axial-attention block yourself and use it in place of full 2-D self-attention (or a convolution) in a small model on your row/column-structured images.

Required original contribution. Implementing the block is the starting point, not the finish line. Benchmark *both* accuracy and compute (FLOPs or wall-clock) of your axial-attention model against a full-2-D-self-attention model and a CNN baseline, as image size (H, W) grows — to empirically test the paper's efficiency claim yourself; or implement the paper's *position-sensitive* variant (learned relative-position terms inside each axial attention) and measure what it buys you; or apply axial attention to a 2-D-structured task of your own choosing and justify why row/column factorization fits it.

Report. Your axial-attention implementation, the accuracy-vs-compute comparison as image size grows, and a look at what the row-attention and column-attention maps each pick up on.

Discussion & observation (*answer these in your report*)

1. A pixel that shares neither its row nor its column with another pixel cannot reach it in a single row-then-column attention step. Can information still flow between them at all, and if so, after how many stacked axial-attention blocks? What does this imply about how the *receptive field* of a stack of axial blocks grows, compared to a single full-attention layer that connects everything immediately?
2. Row/column factorization is a natural fit for OCR because text is laid out in rows and columns of characters. Propose a *different* data modality or task where some other, non-row/column factorization of attention would be the natural choice (e.g. time vs. frequency in an audio spectrogram, space vs. time in video, or the three axes of a volumetric medical scan), and justify why that factorization matches the structure of the data.

Stretch (bonus). Implement the full position-sensitive axial-attention design from the paper and compare it to your simpler row-then-column version.

5. Suggested Report Outline and Submission Checklist

Report outline (2–6 pages per project) 1. **Problem & data** — what you are solving and the dataset.

2. **Method** — your model and training setup.

3. **Original contribution** — the hypothesis and what you designed (the graded core).

4. **Experiments** — baselines, metrics, ablations.

5. **Results & analysis** — honest findings, including where it fails.

6. **Discussion & observation** — your answers to the card’s discussion questions.

7. **Limitations & next steps.**

Before you submit

- ☐ My chosen projects total at least 100 points across ≥ 3 **families** (the two special bonus challenges do not count toward this).
- ☐ Each project has a clear **original contribution**, not just a pretrained run.
- ☐ I answered the **Discussion & observation** questions for *every* project I attempted.
- ☐ Code runs end-to-end from the README with fixed seeds.
- ☐ Metrics are appropriate (per-class / precision-recall where classes are imbalanced).
- ☐ All models, datasets, and code sources are cited.
- ☐ (Optional) A working demo is included for the bonus.

Project scoreboard

Code	Project	Difficulty		Points
RNN-1	Multivariate weather forecasting	Core		25
TF-1	Transformer text classification	Core		25
ViT-1	ViT fine-tune + CNN baseline	Core		30
AE-1	ECG anomaly detection	Foundational		20
VAE-1	VAE generative + latent space	Core		30
TF-2	EN ↔ FA neural machine translation	Advanced		35
ViT-2	Vision Transformer from scratch	Advanced		35
VAE-2	Conditional / β -VAE	Advanced		35
Total available (counts toward the 100)				235
Needed for full score				100
RL-1	Steering an LM with RL (RLHF-lite)	Bonus	Chal- lenge	25
ATT-1	Axial attention for row/column images	Bonus	Chal- lenge	25
Special bonus challenges (independent of the 100)				50

Good luck — build something you would be proud to demo.